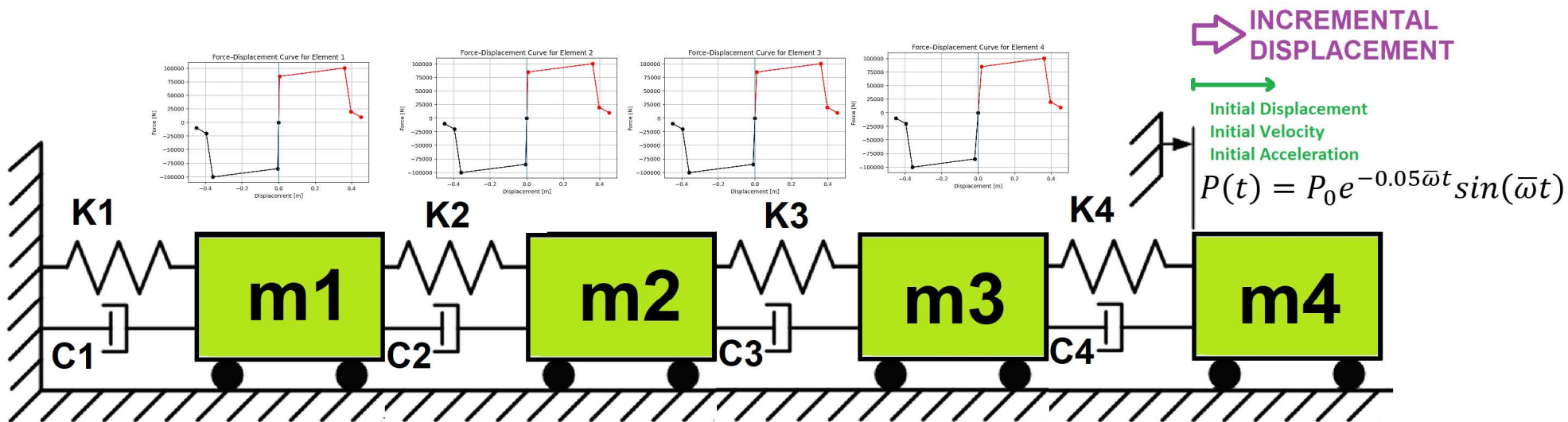


>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

COMPREHENSIVE EVALUATION OF THE EQUIVALENT VISCOUS DAMPING RATIO OF A NONLINEAR MDOF SYSTEM UNDER MULTIPLE LOADING PROTOCOLS

THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)



$$\text{Structural Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

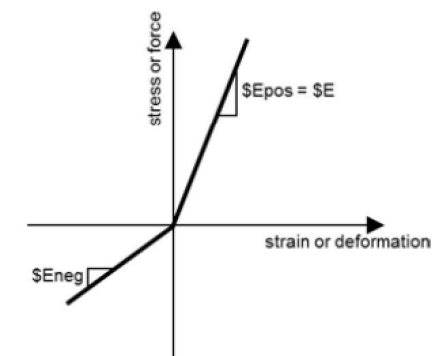
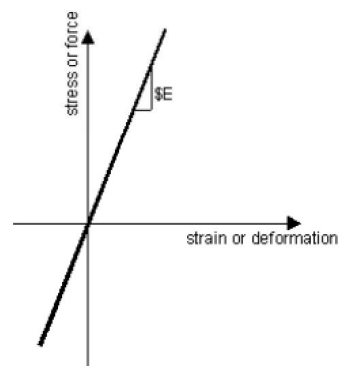
Δ_d = Lateral Displacement from Dynamic Analysis

Δ_y = Lateral Yield Displacement from Pushover Analysis

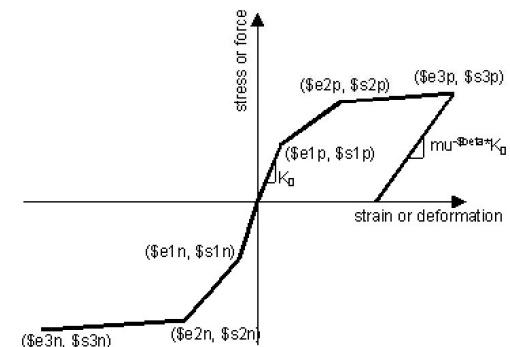
Δ_u = Lateral Ultimate Displacement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

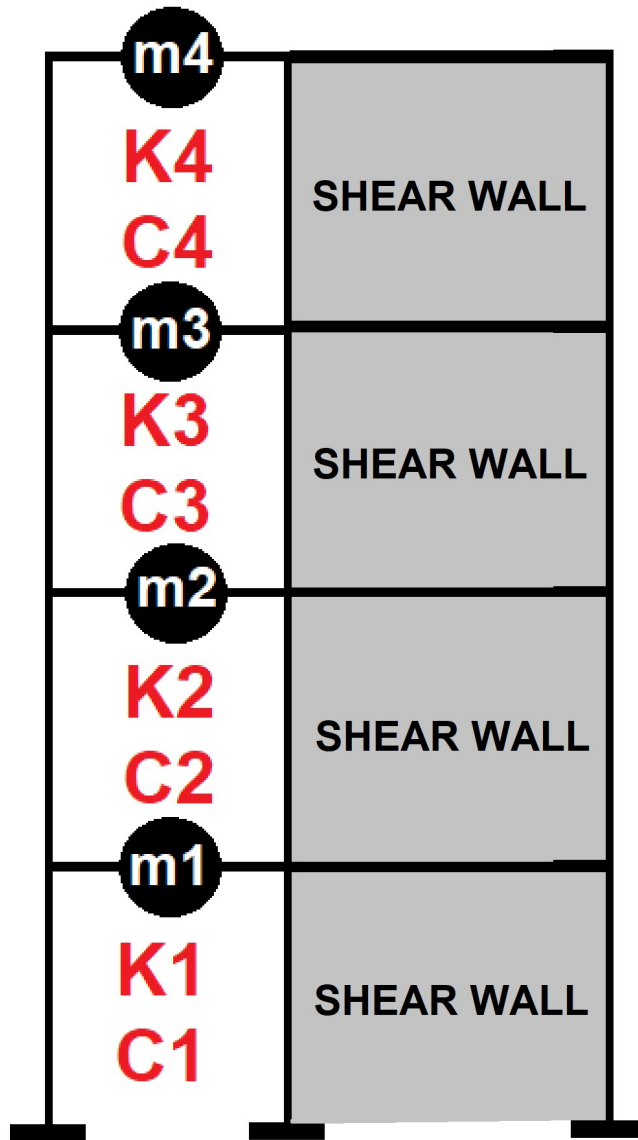
$$m\ddot{u} + c\dot{u} + ku = P(t)$$



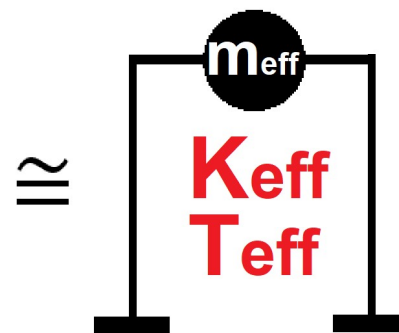
ELASTIC MATERIAL



INELASTIC MATERIAL

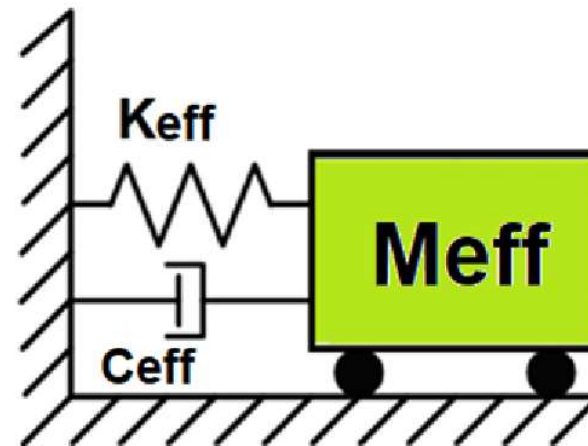


Displacement-based seismic analysis transformation, converting a multi-degree-of-freedom (MDOF) system into an equivalent single-degree-of-freedom (SDOF) system for seismic assessment



$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i} \quad m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d} \quad T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$



Energy quantities and the damping formula

- **Dissipated energy** E_d – the area enclosed by the hysteresis loop. This is the net work done per cycle and represents the energy that must be removed from the system to return it to the original state. Physically, it corresponds to plastic yielding, friction, cracking, or other irreversible mechanisms.
- **Maximum strain energy** E_s – defined here as

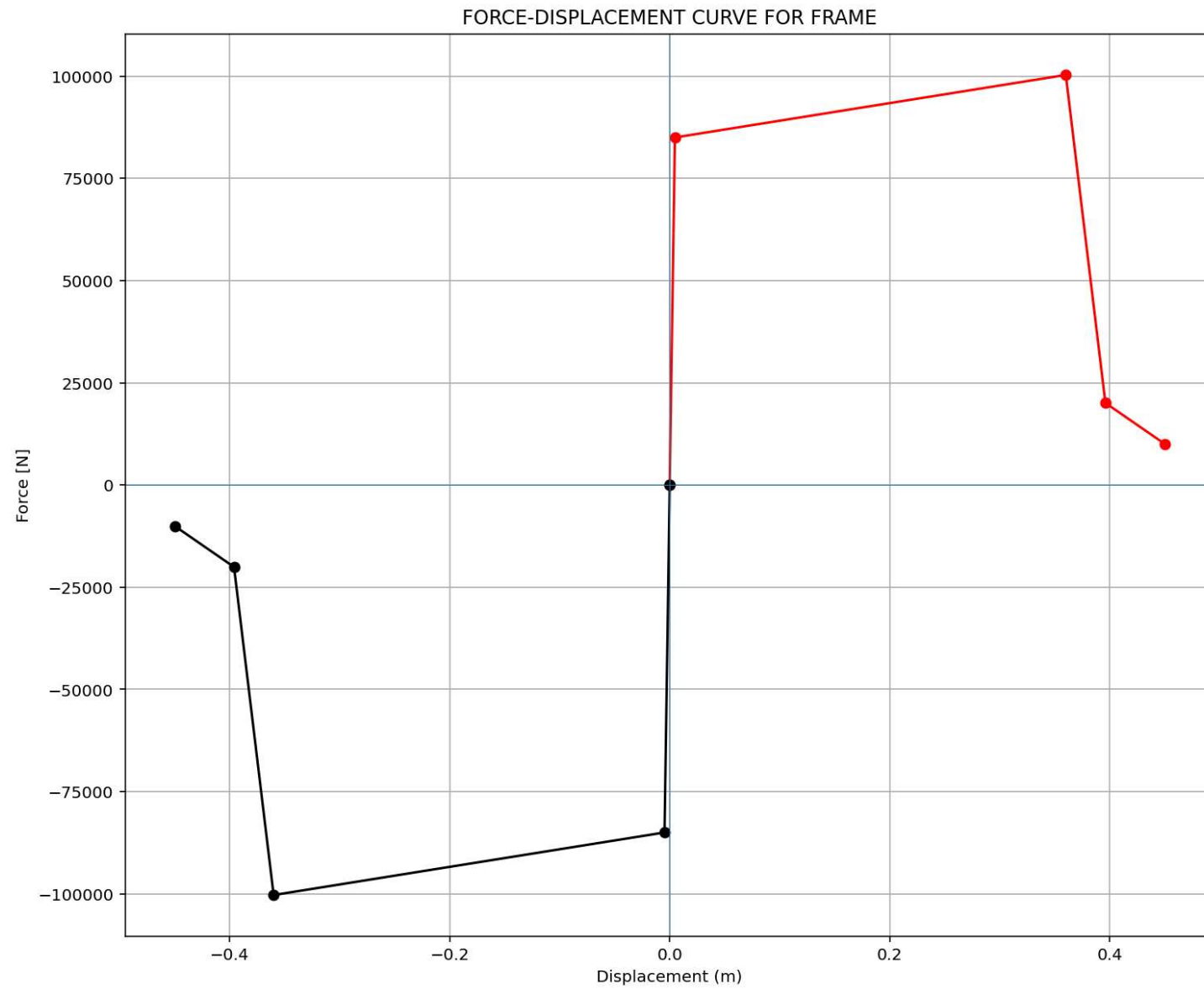
$$E_s = \frac{1}{2} F_{\max} u_{\max}$$

where $u_{\max} = \max |u|$ is the peak absolute displacement in the loop and $F_{\max}$ is the absolute value of the force **at that same displacement**. This definition assumes a **secant stiffness** $k_{\text{sec}} = F_{\max}/u_{\max}$ and represents the elastic strain energy stored at peak displacement if the system were linear with that stiffness.

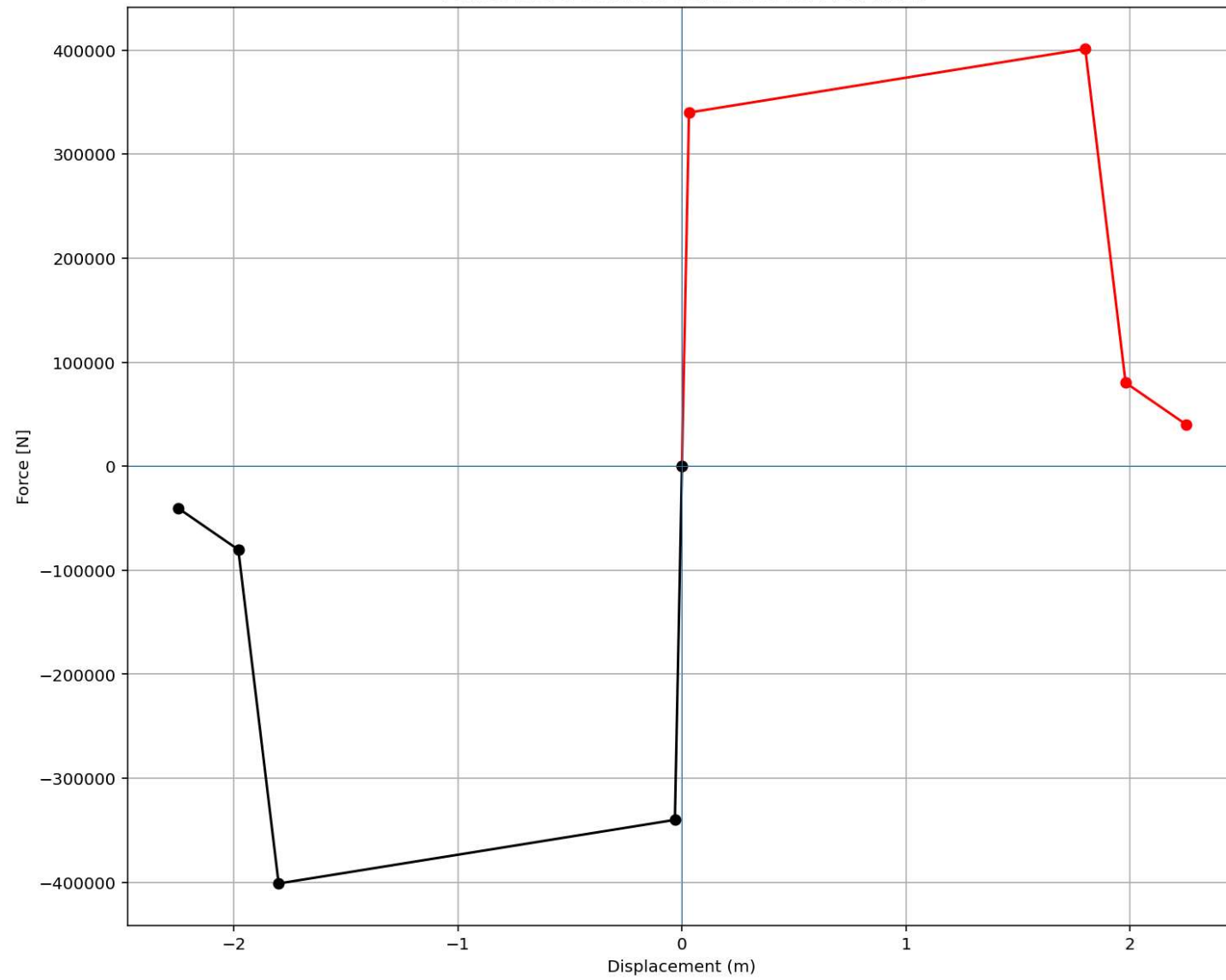
The equivalent viscous damping ratio is then:

$$\xi_{eq} = \frac{100 E_d}{4\pi E_s} \quad [\%]$$

This stems from equating E_d to the energy dissipated per cycle by a linear viscous damper, $E_d = \pi c \omega u_{\max}^2$, while $E_s = \frac{1}{2} k u_{\max}^2$; substituting $\xi = c/(2m\omega)$ and $\omega^2 = k/m$ yields the familiar expression $\xi = E_d/(4\pi E_s)$.



FORCE-DISPLACEMENT CURVE FOR SHEAR-WALL



Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._ANA\COMPUTE_EFFECTIVE_PROPERTIES_FUN_PUSHOVER.py

P(t)_MDOF_TWO_8_ANA.py x COMPUTE_EFFECTIVE...ES_FUN_PUSHOVER.py x

```
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PROCEDURE WITH P
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEEI (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 """
15
16 -----
17 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
18 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
19 displacement_X_3 = np.array(list(node_displacements.values())[3]) # DOF 04
20 displacement_X_4 = np.array(list(node_displacements.values())[4]) # DOF 05
21
22 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
23 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
24 ele_force_03 = np.array(list(ele_force.values())[2]) # ELEMENT 03
25 ele_force_04 = np.array(list(ele_force.values())[3]) # ELEMENT 04
26
27 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
28 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
29 STIFF_X_3 = np.abs(ele_force_03 / displacement_X_3)
30 STIFF_X_4 = np.abs(ele_force_04 / displacement_X_4)
31
32 MX2 = (MASS[0] * np.square(displacement_X_1) +
33        MASS[1] * np.square(displacement_X_2) +
34        MASS[2] * np.square(displacement_X_3) +
35        MASS[3] * np.square(displacement_X_4))
```

Effective Displacement - Median: 0.14592

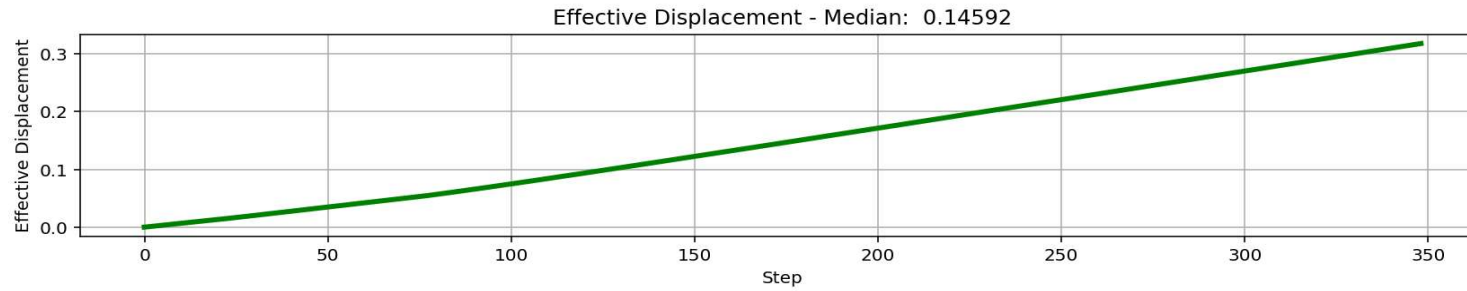
Effective Mass - Median: 4905.18619

Effective Stiffness - Median: 7424871.58691

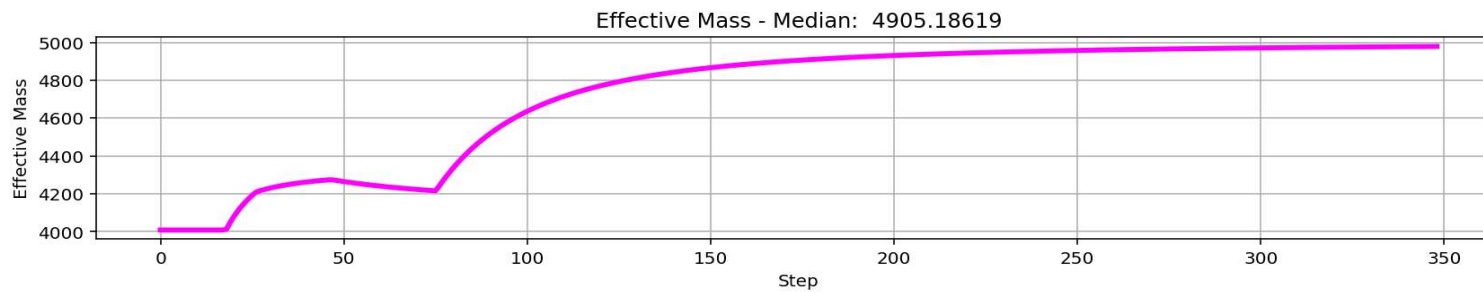
Effective Period - Median: 0.16150

[Python Console] Files Help Variable Explorer Debugger Plots History

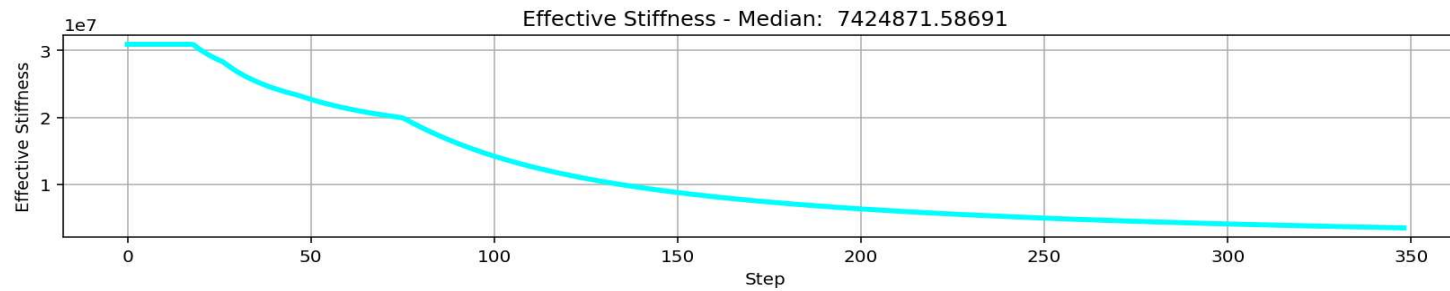
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 49%



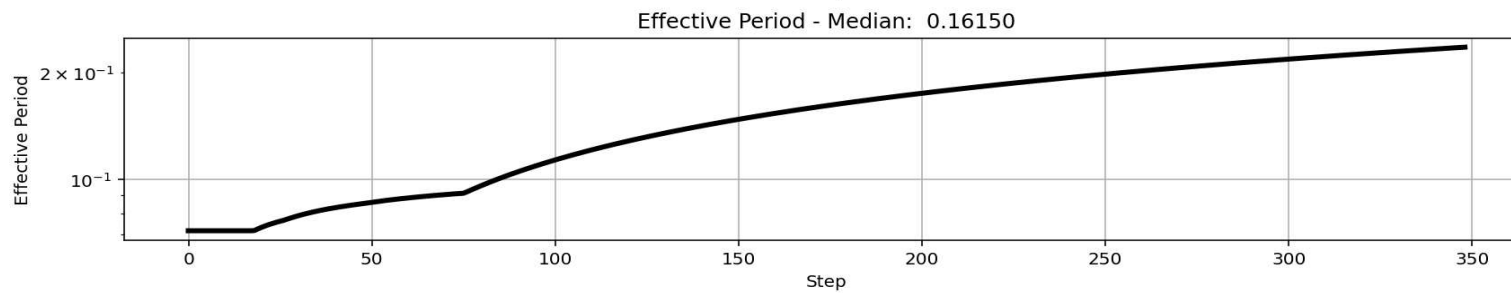
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$



$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$



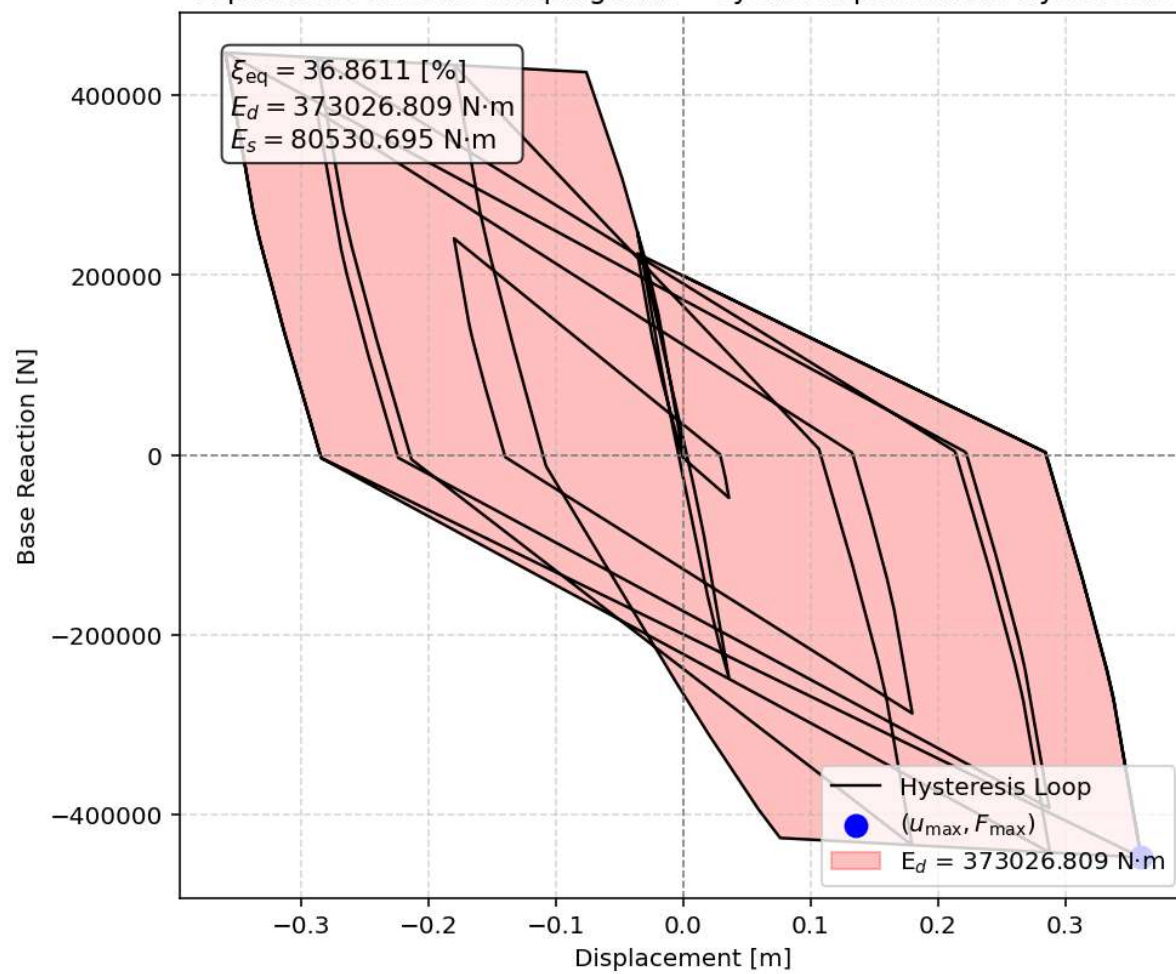
$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$



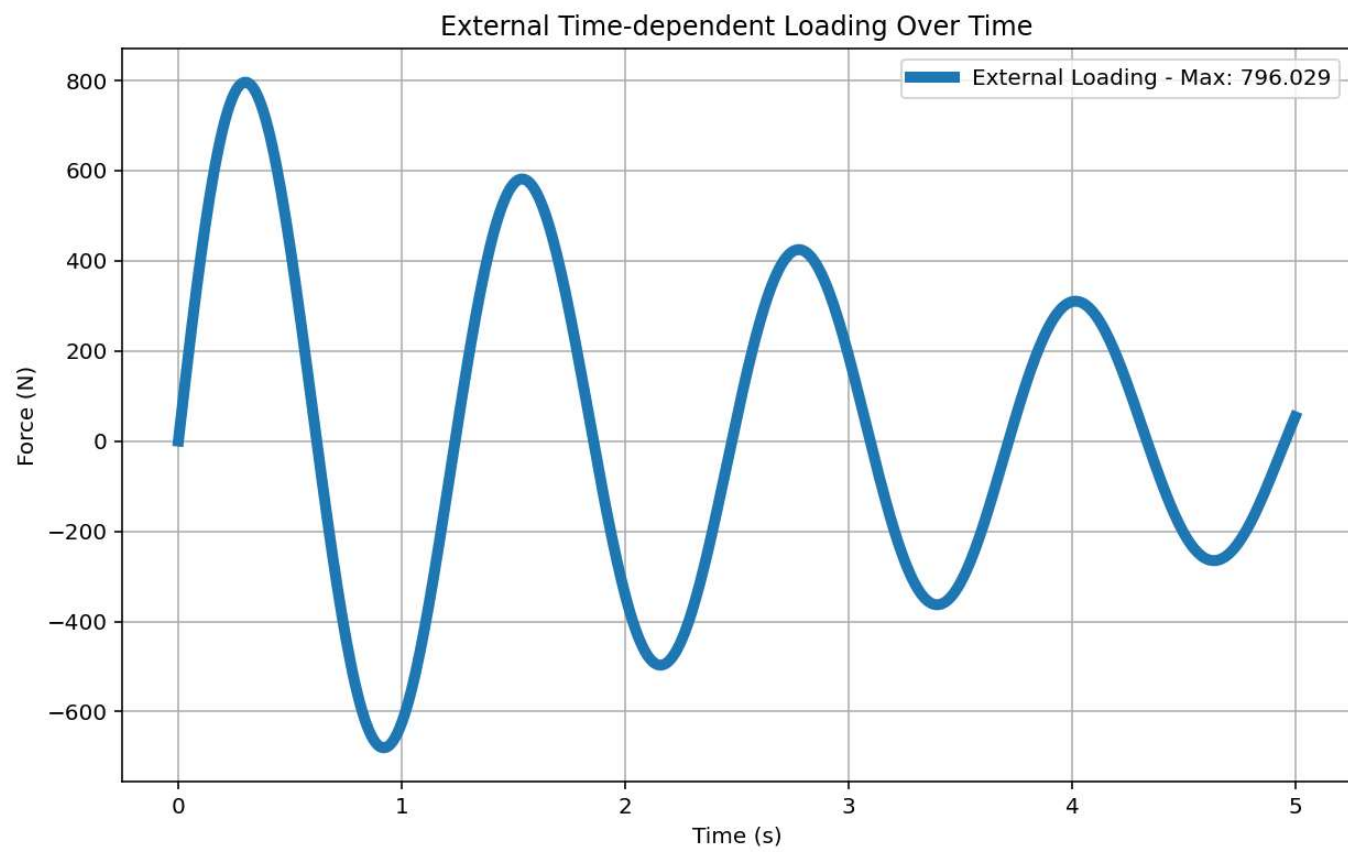
$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

CYCLIC DISPLACEMENT ANALYSIS

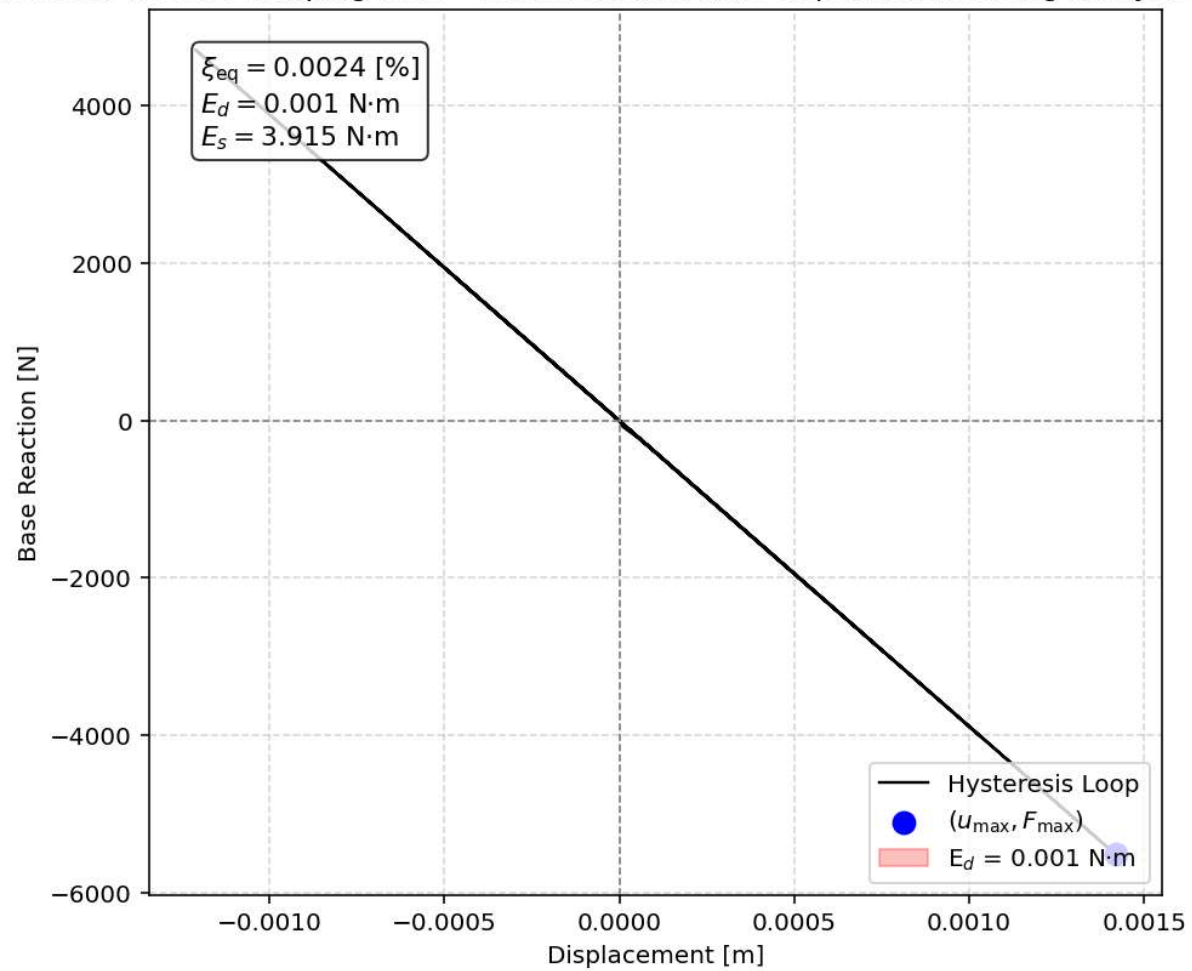
Equivalent viscous damping ratio - Cyclic Displacement Hysteresis



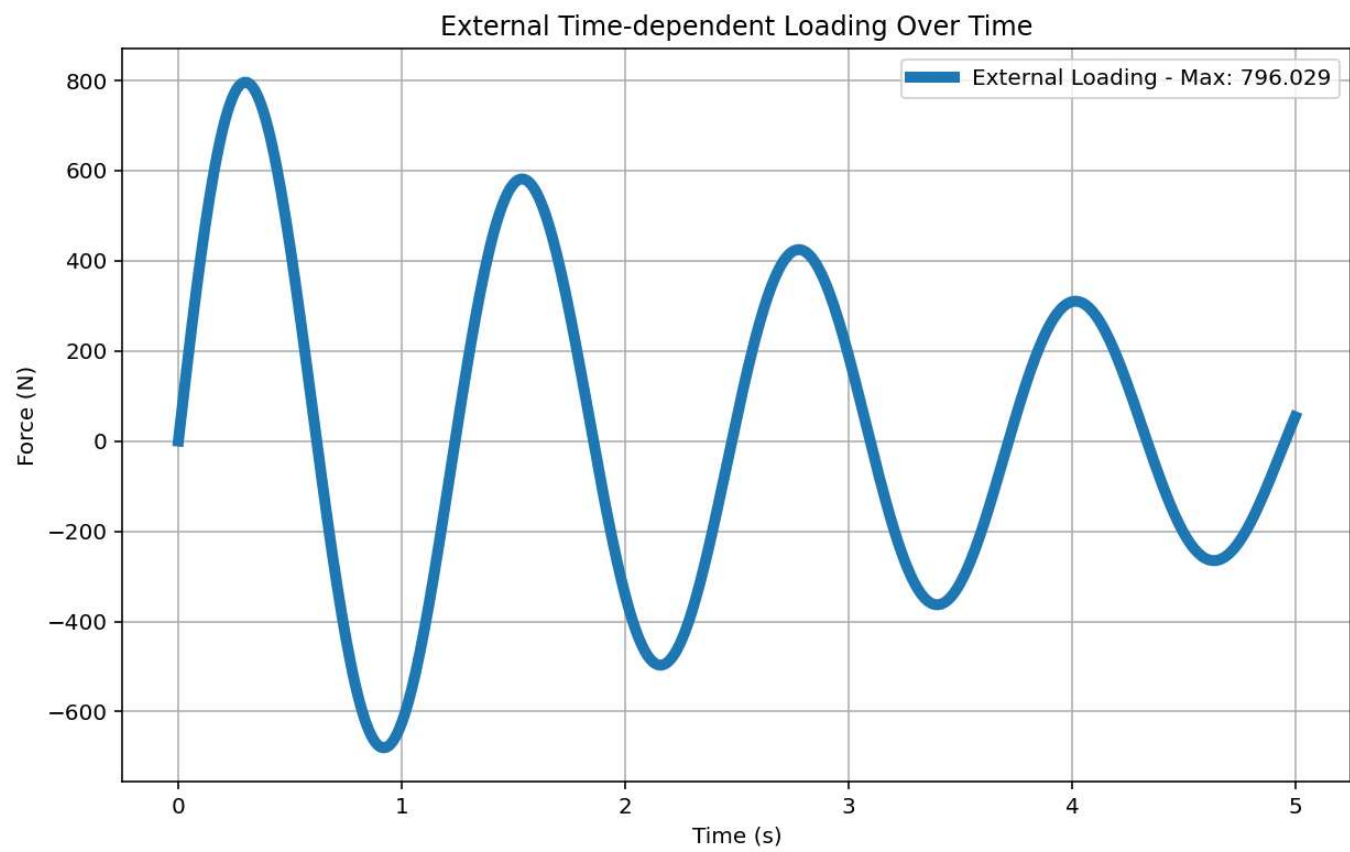
STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS



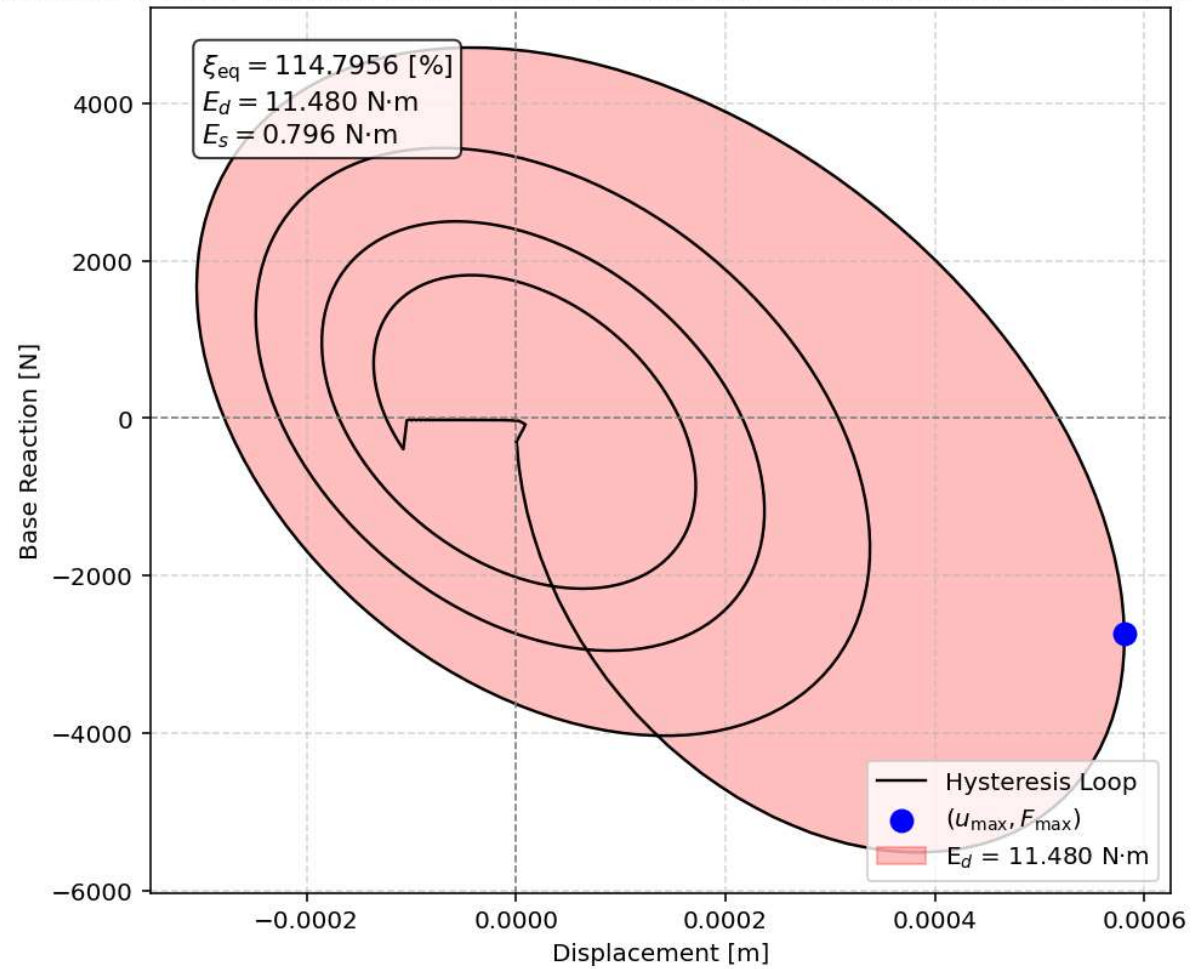
Equivalent viscous damping ratio - Static External Time-dependent Loading Analysis Hysteresis



DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

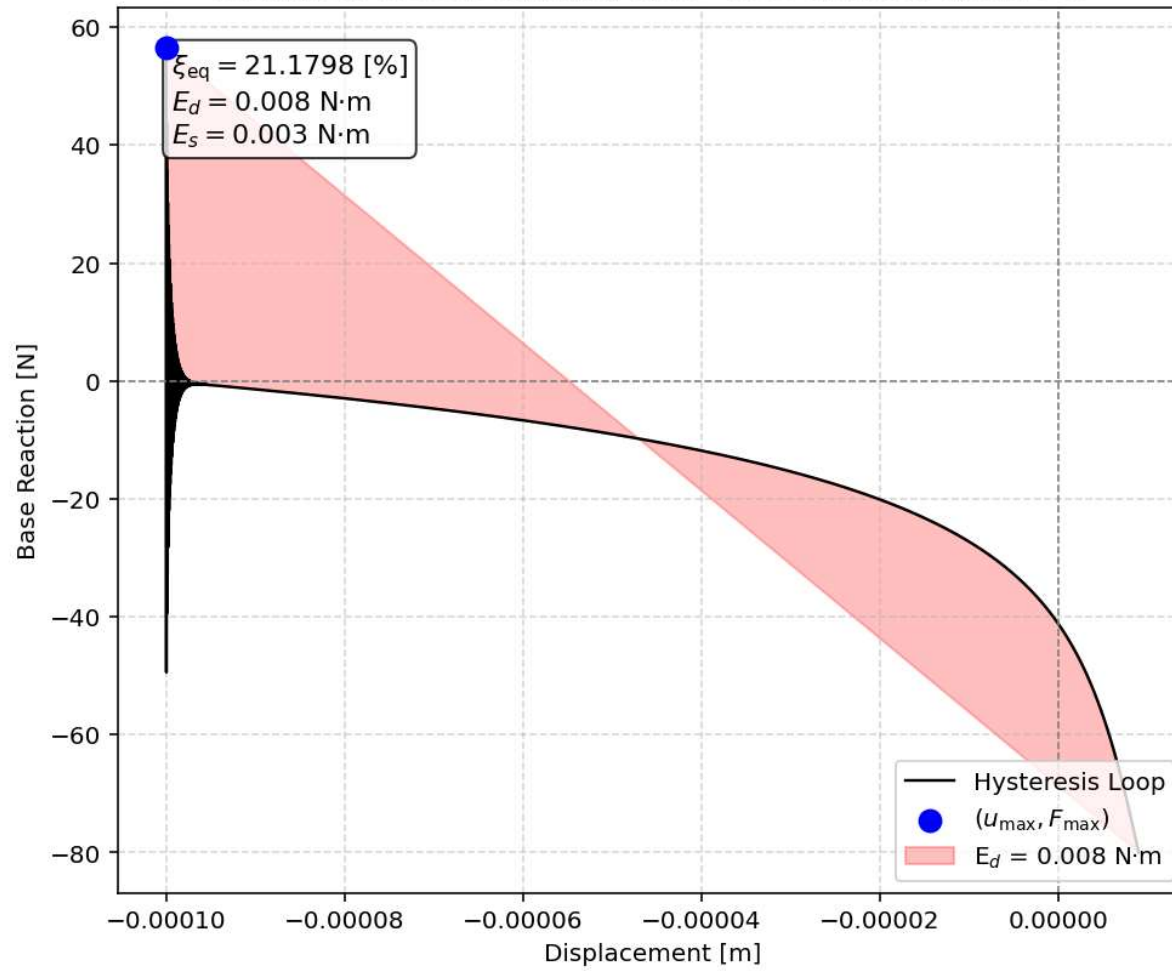


Equivalent viscous damping ratio - Static External Time-dependent Loading Analysis Hysteresis



FREE-VIBRATION ANALYSIS

Equivalent viscous damping ratio - Free-vibration Hysteresis



SEISMIC ANALYSIS

Equivalent viscous damping ratio - Seismic Hysteresis

